EE 542 – Laboratory #3: AWS Tutorial and TCP Congestion Control Modification

Name: Jacinta Rose Menezes

Name: Yichen Zheng

Name: Junjun Liu

Abstract

In this lab Part 1 and Part 2, we focused on learning how to use Amazon Web Services (AWS) by setting up a Virtual Private Cloud (VPC) environment. The tasks included creating a new VPC with a /16 subnet, adding multiple /24 subnets for clients, server, and public access, and configuring an Internet Gateway for external connectivity. We also explored key AWS networking components such as DHCP option sets, route tables, and network ACLs. We also deployed our file-transfer system (from LAB02) to AWS EC2 and tested it under different network configurations similar to LAB02.

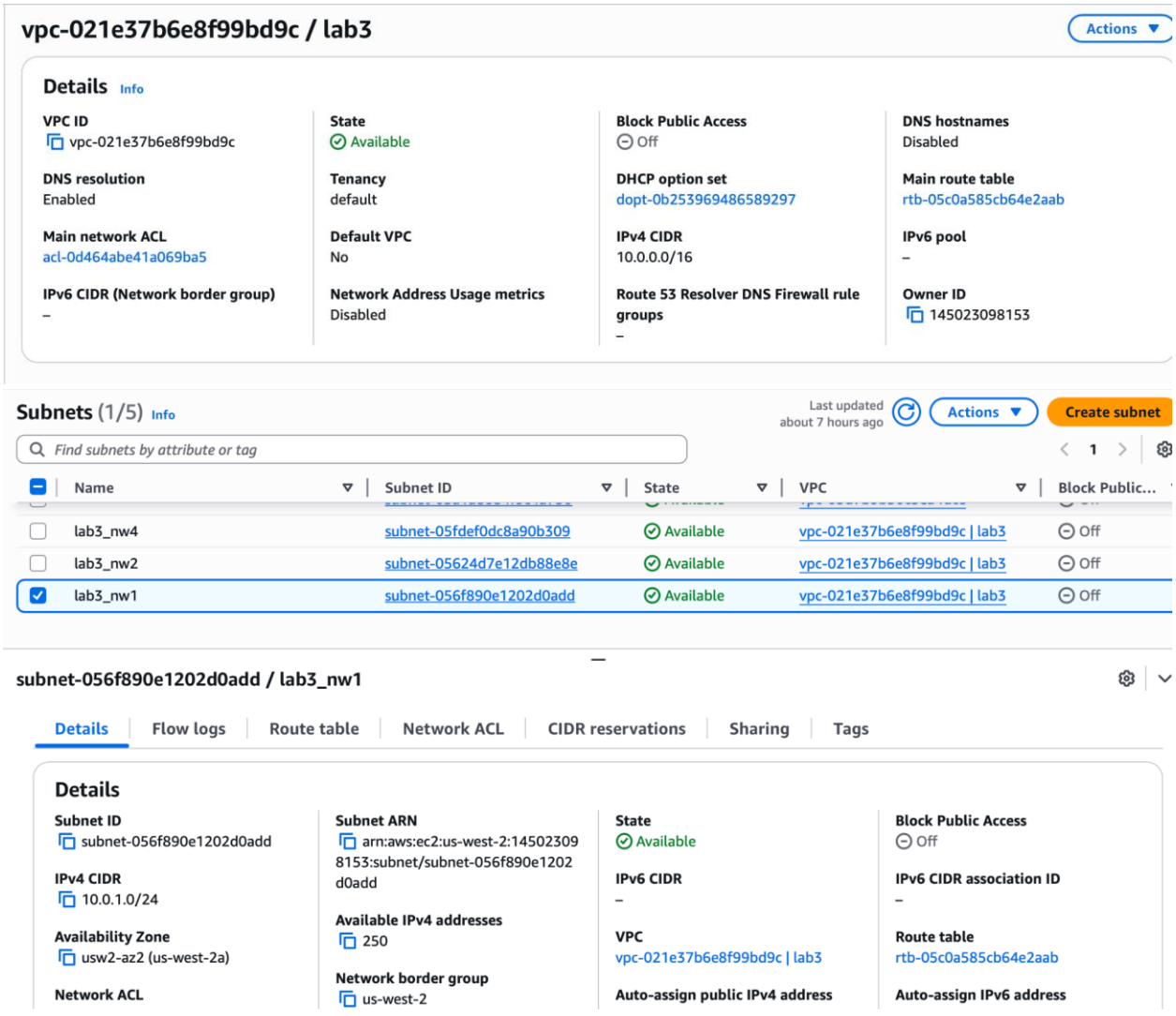
In Part 3 we built a client–server file transfer system based on TCP using the Google BBR congestion control kernel. The main goal was to get good throughput even when the network had high delay or packet loss, while still keeping the system simple to use.

On the client side, our program splits the file into smaller chunks and sends them in parallel over multiple TCP connections. Each thread handles one chunk, identified by its offset and size. This setup lets BBR's pacing keep the flows efficient and helps fully use the available bandwidth. A small control channel is used to send the total file size and a final "done" signal, so the server knows when the transfer is complete.

On the server side, each connection is handled by its own thread. Chunks are written into the correct positions of a shared buffer until the full file is ready, and then the server writes it to disk and sends back an acknowledgment. By combining parallel threads with BBR, the system keeps good performance without needing any extra retransmission logic beyond what TCP already provides.

Part 1: Using Amazon Web Services Cloud Instances (Tutorial)

VPC Part:





Questions:

1.Also, you would have to change the network interface adapter settings of "Source/Destination check" to allow traffic to pass over the VyOS router. Make this setting to Disable. What does this setting do? Explain.

If the check is not disabled, AWS drops packets that don't match the instance's own IP. It allows the VyOS router to act as a middle hop, forwarding traffic between client and server subnets.

2.How is Amazon able to convert your public IP and reach your private IP on the interface?

AWS assigns each instance a private IP inside the VPC and associates a public Elastic IP to its network interface and uses 1:1 NAT mapping.

3.Can you answer why SSH goes down if the IP is changed on that interface? What can be used in VyOS VM to start dhclient in VyOS VM?

When changing the IP of the interface which already has Elastic IP, the SSH session breaks because the network path changes — the session was established with the old IP, and the new IP invalidates the TCP connection.

4.Next step is to install iperf, iperf3 on client, server VM. You would observe that VMs are not able to contact the internet. Why is this? Explain. How can you correct it? To solve it, look at /etc/resolvconf/resolv.conf.d/base file. You would also have to do "sudo apt update". What does this command do?

Because DNS is not set up for VM, it can be solved by adding valid resolvers (8.8.8.8 or AWS provided DNS) in /etc/resolvconf/resolv.conf.d/base. *sudo apt update* refreshes the list of available package versions otherwise the installation might fail.

5.Execute: iperf3 -s on server VM and iperf3 -u -c -b on client VM To test the bandwidth of your connection over UDP. Explore TCP throughput measurement also by using iperf. What is the max throughput and bandwidth you were able to achieve? Why is there any difference in measurement between these two readings over the same link?

UDP can achieve almost line-rate throughput because it has no congestion control or retransmission overhead.
TCP throughput is lower most of the time because it includes congestion control, reliability (ACKs, retransmissions), and flow control.

6.Now, set: sudo tc qdisc add dev eth0 root netem delay 100ms. On the client, server VM, then test the bandwidth again. Were there any changes in readings? If there is a change, why is it? (add -> when first time creating, change -> to update).

Bandwidth measurements drop because the Bandwidth-Delay Product (BDP) increases — TCP must wait longer for ACKs, so the effective throughput is reduced.

7.Execute: sudo tc qdisc add dev eth0 root tbf rate 100mbit latency 1ms burst 9015 Then run step 21 again. What are your observations? What is the significance of the above command, and how does it affect performance?

The command enforces a rate limit of 100 Mbps with a small latency and burst buffer (1ms and 9015).
Effect: Throughput is capped at 100 Mbps regardless of link capacity. This model's ISP or cloud bandwidth throttling.
Observation: Throughput stabilizes at ~100 Mbps but latency/packet drops increase if traffic exceeds the burst buffer.

8.Execute sudo ethtool -s eth0 speed 10 Did you get any error, and why is it not allowed in AWS? Explain.

Yes, we cannot change the physical NIC speed in AWS because AWS does not give hardware control permission to guest OS.
In part1 we created a VPC with four subnets (client, server, and two external). An Internet Gateway was attached for outside access, and a default route (0.0.0.0/0) was added.
Elastic IPs provided public connectivity via AWS's 1:1 NAT mapping. To let the VyOS instance forward traffic, we disabled Source/Destination Check. We know that changing an Elastic-IP-bound interface's IP immediately breaks SSH, and restarting DHCP with dhclient restores it.

Commands and Debugging

- **ifconfig / ip route** to configure interfaces and routes.
- **tcpdump** confirmed packets crossing VyOS.
- **ip route show** to verify routing table
- **ping <server-IP>** and **ping -I eth1 <ip>** to test per-interface reachability
- **dhclient eth1** to request new DHCP lease
- **sudo apt update** resolved package install errors.

iperf Measurements and tc Commands

1.Baseline (no delay/loss)

- TCP throughput approached line-rate (~94 Mbps).
- UDP achieved a near send-rate (~98 Mbps) with negligible loss.

2.100ms delay via tc qdisc add dev eth0 root netem delay 100ms

- TCP throughput dropped due to larger Bandwidth-Delay Product and slower ACK feedback (~75 Mbps).
- UDP maintained ~100 Mbps send rate, with slightly higher jitter.

3.10% loss via tc qdisc change dev eth0 root netem loss 10%

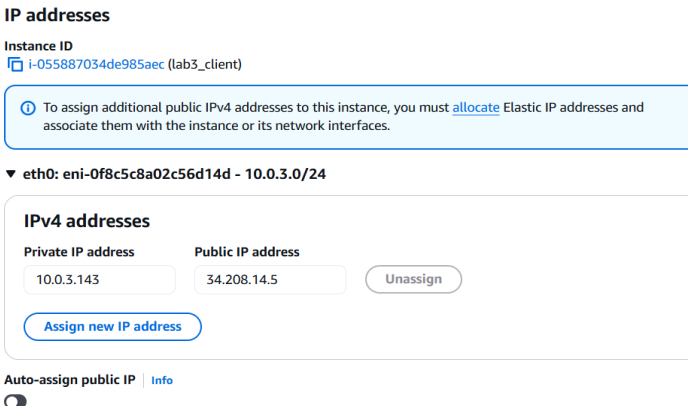
- TCP collapsed to ~12 Mbps because of retransmissions and congestion backoff.
- UDP still transmitted at ~100 Mbps, but ~10% of packets were lost, confirming the configured rate.

4.Rate limiting with Token Bucket Filter (tc qdisc add dev eth0 root tbf rate 100mbit latency 1ms burst 9015)

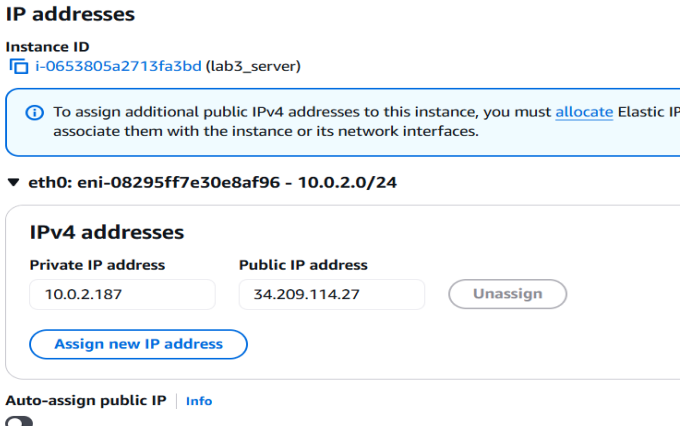
- Both TCP and UDP capped close to 100 Mbps.
- Excess UDP traffic was dropped, while TCP adapted smoothly.

Part 2: Fast and Reliable File Transfer in the Cloud

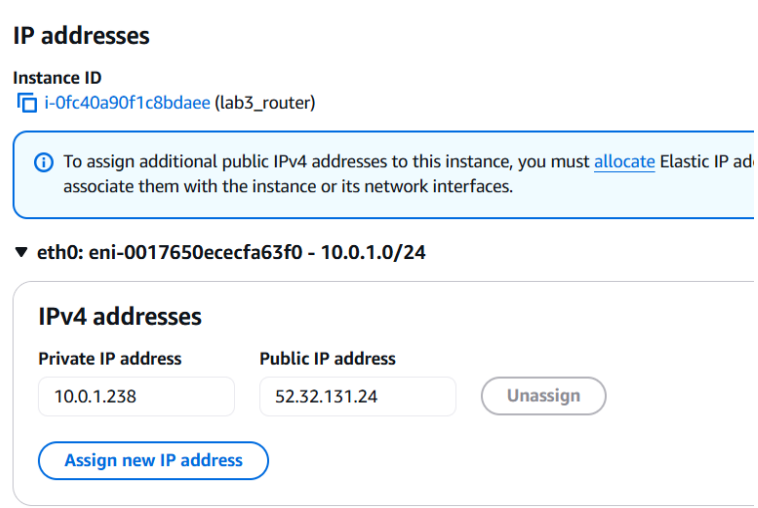
Client:



Server:



Router:



The results of this are shown in the demonstration video.

Part 3: Fast and Reliable File Transfer in the Cloud (THE MAIN PART)

Traditional TCP congestion control algorithms like Reno and Cubic are fundamentally loss-based: they interpret packet loss as congestion and sharply reduce the congestion window. This leads to poor throughput on paths with high bandwidth-delay product and non-congestion losses (e.g., wireless environments).

We select **BBR (Bottleneck Bandwidth and RTT)**, it is a congestion control algorithm developed by Google, BBR does not rely on packet loss as a congestion signal. Instead, it builds a model of the network path. We use BBR in this lab because it has the following features:

- **Model-based instead of loss-based**
    - Estimates the bottleneck bandwidth and minimum RTT (RTprop).
    - Sends at a pacing rate ≈ BtlBw × RTprop, targeting the bandwidth-delay product.
      BtlBw*: Bottleneck Bandwidth — the maximum delivery rate of the path.*

      RTprop*: Round-trip propagation time — the minimum RTT observed, representing baseline delay without queueing.*
    - Avoids drastic cwnd reductions when losses are due to noise or random drops.
- **Higher throughput in lossy environments**
    - Ideal for our lab setup with induced losses and varied delays.
    - Unlike Reno, BBR maintains near-full utilization without collapsing cwnd.

- **Widely deployed in industry**
    - Used in Google services, YouTube, and QUIC (HTTP/3).
    - Makes the lab work realistic and relevant to real-world networking.
- **Drop-in replacement**
    - Available in modern Linux kernels.
    - No kernel coding required; we can focus effort on **application-layer protocol design** (multi-threaded client/server).

System Structure Design

Server Design:

Listener Socket:

- The server listens for incoming client connections.
- Each new connection is handled in a dedicated worker thread.

Chunk Reception:

- Each connection reads the (offset, size) header.
- If size > 0, the server stores the chunk at the given offset in a shared file buffer.
- If size = 0, the server interprets it as a DONE marker.

File Reassembly:

- The server maintains a global progress tracker.
- Once all expected bytes are received, the file buffer is flushed to disk.

Completion Signaling:

- After the file is fully reconstructed, the server sends an **ACK_DONE** back to the client.

Client Design:

Control Connection:

- A dedicated TCP connection sends the **file size** at the beginning.
- This allows the server to pre-allocate buffers or prepare output files.

Worker Threads:

- The client spawns **multiple threads**, each opening its own TCP connection.
- Each thread is assigned a **sub-chunk** of the file, defined by its (offset, size).

Data Transmission:

- Each thread first sends a small header (offset, size) (two uint64_t values).
- The actual file bytes are then streamed immediately after the header.

Completion Signaling:

- When all chunks are sent, the client transmits a **DONE marker** (offset=0, size=0).
- The client then waits for the server to send back an **ACK_DONE** before terminating.

Result

Set up Vyos:

```
vyos@vyos:~$ sudo tc qdisc add dev eth0 root handle 1:0 tbf rate 100mbit latency
 0.001ms burst 901555
vyos@vyos:~$ sudo tc qdisc add dev eth1 root handle 1:0 tbf rate 100mbit latency
 0.001ms burst 901555
vyos@vyos:~$ sudo tc qdisc add dev eth2 root handle 1:0 tbf rate 100mbit latency
 0.001ms burst 901555
vyos@vyos:~$ sudo tc qdisc add dev eth0 parent 1:1 handle 10: netem delay 100ms
drop 20%
vyos@vyos:~$ sudo tc qdisc add dev eth1 parent 1:1 handle 10: netem delay 100ms
drop 20%
vyos@vyos:~$ sudo tc qdisc add dev eth2 parent 1:1 handle 10: netem delay 100ms
drop 20%
```

Running:

```
client@client:~$ ./client 192.168.10.100 9000 ./bigfile.bin 64
[03:42:36.179] [CLIENT] Total file size: 1073741824 bytes
```

```
server@server:~$ ./server  9000 output9.bin
[03:42:33.332] [SERVER] Listening on port 9000...
[03:42:36.302] [SERVER] Expecting file size 1073741824 bytes
[03:45:07.084] [SERVER] File complete on disk: 1073741824 bytes

[03:45:07.084] [SERVER] Total received (counter): 1073741824 bytes in 153.851 sec. Throughput: 55.8327 Mbps
```

The throughput reaches 55.83 Mbps over a network that is configured to be 100 Mbps with 200ms RTT latency (100ms delay each way) and 20% packet loss (bi-directional).

Verification:

```
client@client:~$ md5sum ./bigfile.bin
cd573cfaace07e7949bc0c46028904ff  ./bigfile.bin
```

```
server@server:~$ md5sum  output9.bin
cd573cfaace07e7949bc0c46028904ff  output9.bin
```

Set up AWS:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Publ
lab3_client	i-055887034de985aec	Stopping	t3.micro	Initializing	View alarms +	us-west-2a	–
test	i-0dff9192753a0d7dd	Terminated	t3.micro	–	View alarms +	us-west-2a	–
lab3_server	i-0653805a2713fa3bd	Stopped	t3.micro	–	View alarms +	us-west-2a	–
Lab3_build	i-0fc40a90f1c8bdaee	Stopping	t3.micro	–	View alarms +	us-west-2a	–

The results of this are shown in the demonstration video.

Workflow

Client Workflow

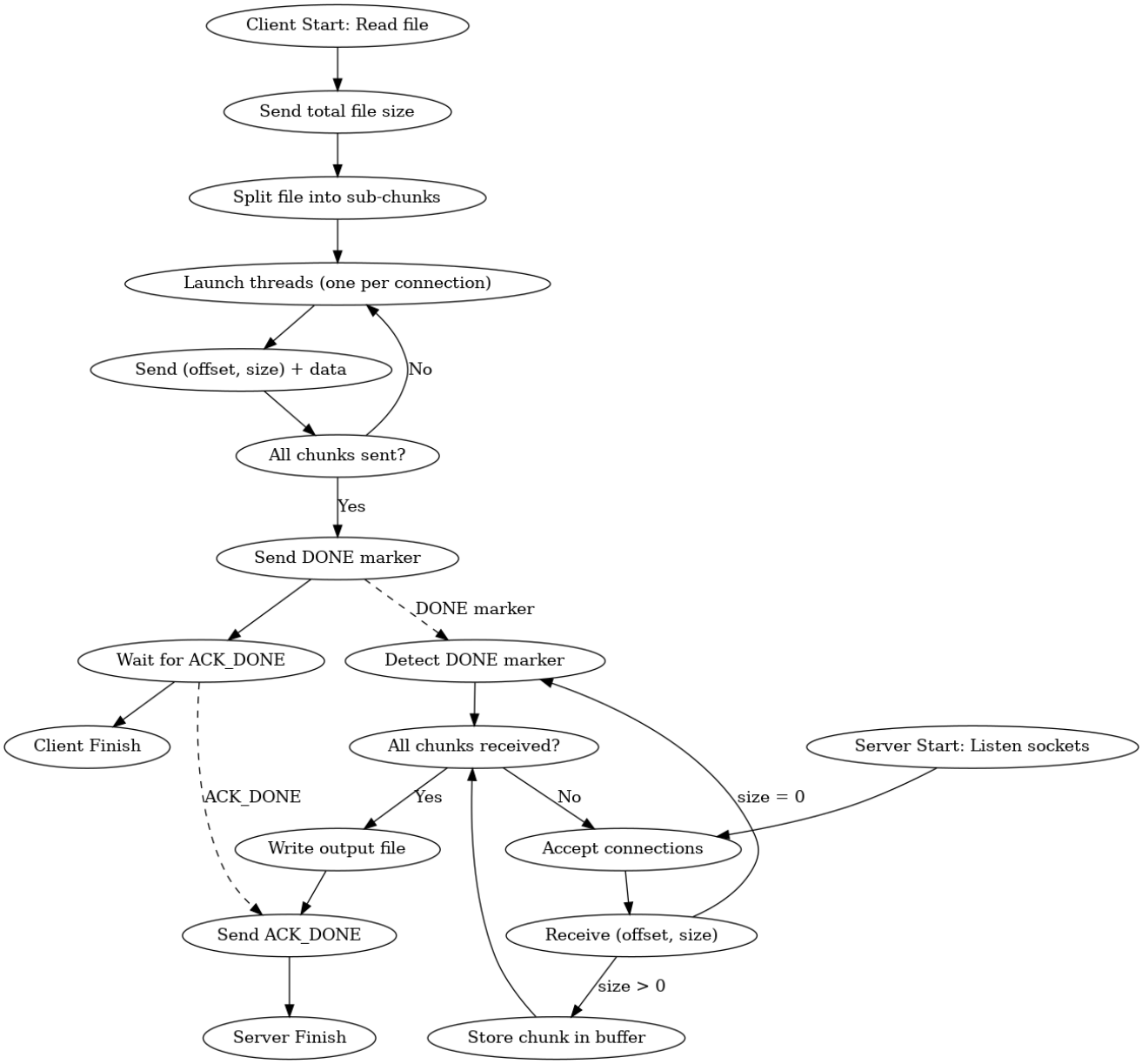
1. **Initialization (main)**
    - Parse arguments, open the input file, compute total size.
    - Open a control connection → send total file size → close it.
2. **Chunk Assignment (send_chunk)**
    - Split file into chunks for each thread.
    - Each thread further splits into smaller sub-chunks (streams).
3. **Sub-chunk Transmission (send_subchunk)**
    - Connect to the server.
    - Send header: (offset, size) of the sub-chunk.
    - Stream file bytes from disk → socket.
    - If it's the very last sub-chunk overall: send a **DONE marker** and wait for **ACK_DONE**.
4. **Progress Monitoring (print_progress)**
    - A dedicated thread tracks and prints how much each stream has sent until all threads complete.

Server Workflow

- **Setup (main)**
    - Create a listening socket.
    - First accepted connection → receive total file size → close.
- **Connection Handling (handle_client)**

  For each data connection:
    - Read header (offset, size).
    - If normal chunk: receive data, place it in the correct position using pwrite.
    - If header is (0,0): treat as **DONE marker**, reply with **ACK_DONE**, and stop accepting new streams.
- **Progress Monitoring (print_progress)**
    - Background thread prints per-connection progress until transfer finishes.

Flow Chart of Protocol Used (Algorithm)



Data Structures Used in Code

Client Side

1. **Subchunk (struct/class)**
    - Fields:
        - uint64_t offset → starting byte position in the file
        - uint64_t size → length of this sub-chunk
    - Used to tell both the client worker thread and the server exactly *which part* of the file is being transferred.
2. **Atomic counters (std::atomic<size_t> sent_bytes)**
    - Tracks how many bytes each thread has successfully written to its socket.
    - Used by the progress thread (print_progress) to show total progress across all connections.
3. **Thread arguments (ThreadArg)**
    - Fields:
        - Subchunk chunk → what part of the file to send
        - int thread_id → for logging/progress display
    - Passed into send_chunk or send_subchunk.

Server Side

1. **ChunkHeader (struct)**
    - Fields:
        - uint64_t offset
        - uint64_t size
    - Sent by the client before each data segment.
    - Server uses this to place received bytes at the right position (via pwrite).
2. **Shared buffer / output file descriptor**
    - The server directly writes into the output file at the correct offset.
    - Synchronization is handled by the kernel (pwrite is atomic at offset), so no extra locks needed.
3. **Atomic counters (std::atomic<size_t> received_bytes)**
    - Tracks progress of all connections.
    - Read by the print_progress thread to report status.

Conclusion

In this lab, our team implemented a custom client–server file transfer system that works on top of TCP with BBR congestion control. Our design used multiple parallel connections, chunk headers, and clear control messages to handle file splitting, transmission, and reassembly. By combining multi-threading with BBR's pacing, the system achieved stable throughput even under higher delays and packet loss conditions, while still keeping the implementation relatively simple.

One of the key lessons from this project was how congestion control matters in practice. Traditional loss-based TCP algorithms like Reno can collapse throughput in lossy networks, while BBR models the path and maintains better performance. We also learned how application-level design choices, like splitting files into sub-chunks and coordinating multiple threads, can directly affect how well the transport layer performs.

Overall, the lab showed me how to bridge kernel-level transport behavior with application-level protocol design. It helped us better understand the trade-offs between reliability, efficiency, and simplicity.